```
"""
================================================================================
==
 ETABS v22 MODEL BUILDER -  5-Story RC Dual System (SMF + Core Shear Walls)
 Muneeb Jadoon |  ASCE 7-22 / ACI 318-19 |  units: kip, ft
================================================================================
==
 HOW TO RUN
   1. Open ETABS 22 (a blank/new instance is fine - the script wipes it).
   2. In VS Code, activate the venv where comtypes is installed.
   3. Run:  python ETABS_Build_5Story_DualSystem.py
   4. The script ATTACHES to the running ETABS. Watch the ETABS window and
      the console. RUN_ANALYSIS is False by default so you can eyeball the
      geometry first; set it True to run.
================================================================================
==
"""
import comtypes.client
import math


RUN_ANALYSIS = False      # set True to run after you have eyeballed the model


# ---------------------------------------------------------------- geometry
XS = [0.0, 24.0, 48.0, 72.0, 96.0]     # gridlines A B C D E  (ft)
YS = [0.0, 24.0, 48.0, 72.0]           # gridlines 1 2 3 4     (ft)
H  = 12.0                              # typical story height (ft)
NSTORY = 5
```

```
ZT = [H*(i+1) for i in range(NSTORY)]   # floor elevations: 12,24,36,48,60
ZALL = [0.0] + ZT

# central core box (ft): X 36->60, Y 24->48, walls 14" thick
CX0, CX1, CY0, CY1 = 36.0, 60.0, 24.0, 48.0
TW = 14.0/12.0                      # wall thickness, ft

# ------------------------------------------------------------------ materials
FC_COL = 5.0*144.0   # 5 ksi  -> ksf   (columns + walls)
FC_BM  = 4.0*144.0   # 4 ksi  -> ksf   (beams + slab)
E5 = 57000.0*math.sqrt(5000.0)/1000.0*144.0   # ksf
E4 = 57000.0*math.sqrt(4000.0)/1000.0*144.0   # ksf
FY = 60.0*144.0      # 60 ksi -> ksf

# ------------------------------------------------------------------ loads (ksf)
SDL_FLR, LIVE_FLR = 0.030, 0.050        # floors: finishes/MEP ; office live
SDL_ROOF, LIVE_ROOF = 0.025, 0.020      # roof
CLAD = 0.25                         # perimeter cladding line load (k/ft)

# ------------------------------------------------------------------ attach
print("="*70)
helper = comtypes.client.CreateObject('ETABSv1.Helper')
helper = helper.QueryInterface(comtypes.gen.ETABSv1.cHelper)
try:
    etabs = helper.GetObject("CSI.ETABS.API.ETABSObject")
    print("Attached to the running ETABS instance.")
```

```
except Exception:
    raise SystemExit("Could not attach to ETABS. Open ETABS 22 first, then re-run.")
sm = etabs.SapModel

def step(msg, ret=None):
    tag = "" if ret is None else f"   [ret={ret}]"
    print(f"  {msg}{tag}")

# ------------------------------------------------------------------ init + grid + stories
# NewGridOnly lays down the labeled grid (A-E x 1-4 at 24 ft) AND the 5 stories
# in one call, so every column lands on a real, named grid intersection.
sm.InitializeNewModel(4)       # 4 = kip_ft_F
sm.File.NewGridOnly(NSTORY, H, H, len(XS), len(YS), 24.0, 24.0)
sm.SetPresentUnits(4)
step(f"Grid created: {len(XS)} x {len(YS)} lines @ 24 ft (A-E / 1-4), {NSTORY} stories @ {int(H)} ft")

# ------------------------------------------------------------------ materials
sm.PropMaterial.SetMaterial("C5", 2)                    # 2 = concrete
sm.PropMaterial.SetMPIsotropic("C5", E5, 0.2, 5.5e-6)
sm.PropMaterial.SetMaterial("C4", 2)
sm.PropMaterial.SetMPIsotropic("C4", E4, 0.2, 5.5e-6)
try:
    sm.PropMaterial.SetOConcrete_1("C5", FC_COL, False, 0, 1, 2, 0.0022, 0.0052, -0.1, 0, 0)
    sm.PropMaterial.SetOConcrete_1("C4", FC_BM , False, 0, 1, 2, 0.0022, 0.0052, -0.1, 0, 0)
except Exception as e:
    step("SetOConcrete_1 flagged (set f'c in GUI if needed): "+str(e))
```

```
try:
   sm.PropMaterial.SetMaterial("A615", 6)              # 6 = rebar
   sm.PropMaterial.SetORebar_1("A615", FY, 90*144.0, FY, 90*144.0, 1, 2, 0.01, 0.09, -0.1, False,
0)
except Exception as e:
   step("Rebar material flagged (define A615 Gr60 in GUI if needed): "+str(e))
step("Materials: C5 (5 ksi), C4 (4 ksi), A615 Gr60")


# ------------------------------------------------------------------ sections
sm.PropFrame.SetRectangle("COL30",  "C5", 2.5, 2.5)       # 30 x 30 in
sm.PropFrame.SetRectangle("BM18X30","C4", 2.5, 1.5)       # 30 deep x 18 wide
# modifiers [Area,As2,As3,J,I22,I33,mass,weight]  (ACI 6.6.3.1: col 0.70, beam 0.35)
sm.PropFrame.SetModifiers("COL30",  [1,1,1,1,0.70,0.70,1,1])
sm.PropFrame.SetModifiers("BM18X30",[1,1,1,1,0.35,0.35,1,1])
# concrete design type: beam designs as a BEAM, column as a COLUMN
try:
   sm.PropFrame.SetRebarBeam("BM18X30","A615","A615",0.1875,0.1875,0,0,0,0)
   step("BM18X30 tagged concrete BEAM (flexural design)")
except Exception as e:
   step("SetRebarBeam flagged (set beam design type in GUI): "+str(e))
try:
   sm.PropFrame.SetRebarColumn("COL30","A615","A615",1,1,0.1667,0,4,4,"#9","#4",0.3333,2,
2,True)
   step("COL30 tagged concrete COLUMN (P-M-M design)")
except Exception as e:
   step("SetRebarColumn flagged (set column design type in GUI): "+str(e))
# wall + slab
```

```
try:
    sm.PropArea.SetWall("WALL14", 1, 1, "C5", TW)         # specified, shell-thin
except Exception as e:
    step("SetWall flagged: "+str(e))
try:
    sm.PropArea.SetSlab("SLAB8", 0, 3, "C4", 8.0/12.0)     # membrane (3) if build maps it so
except Exception as e:
    step("SetSlab flagged - set SLAB8 to Membrane in GUI if it imports as shell: "+str(e))
# wall modifiers (cracked core: bending 0.35, membrane 0.70)
try:
    sm.PropArea.SetModifiers("WALL14", [0.70,0.70,0.70,0.35,0.35,0.35,0.70,0.70,1,1])
except Exception as e:
    step("Wall modifiers flagged: "+str(e))
step("Sections: COL30, BM18X30, WALL14, SLAB8 (modifiers applied)")

# ---------------------------------------------------------------- helpers
def _name(res):
    # comtypes returns all [in,out] params then the ret code last,
    # so the object name is always the second-to-last element.
    if isinstance(res,(list,tuple)):
        return res[-2] if len(res) >= 2 else res[0]
    return res

def add_frame(x1,y1,z1,x2,y2,z2,prop):
    return _name(sm.FrameObj.AddByCoord(x1,y1,z1,x2,y2,z2,"",prop,"","Global"))
```

```
def add_area(xl,yl,zl,prop):
    n = len(xl)
    return _name(sm.AreaObj.AddByCoord(n, xl, yl, zl, "", prop, "", "Global"))


# ---------------------------------------------------------------- columns
ncol = 0
for x in XS:
    for y in YS:
        for s in range(NSTORY):
            add_frame(x,y,ZALL[s], x,y,ZALL[s+1], "COL30"); ncol += 1
step(f"Columns added: {ncol}")


# ---------------------------------------------------------------- beams
peri = []         # perimeter beam names for cladding
nbm = 0
for z in ZT:
    for y in YS:                  # X-direction beams
        for i in range(len(XS)-1):
            nm = add_frame(XS[i],y,z, XS[i+1],y,z, "BM18X30"); nbm += 1
            if y in (YS[0], YS[-1]): peri.append(nm)
    for x in XS:                  # Y-direction beams
        for j in range(len(YS)-1):
            nm = add_frame(x,YS[j],z, x,YS[j+1],z, "BM18X30"); nbm += 1
            if x in (XS[0], XS[-1]): peri.append(nm)
step(f"Beams added: {nbm}  (perimeter tagged: {len(peri)})")
```

```
# ------------------------------------------------------------------ distributed perimeter shear walls
# 8 walls at the building corners: 4 resist X (grids 1 & 4 end bays),
# 4 resist Y (grids A & E end bays). Each 24 ft long, one pier each.
# This spreads base shear over 4 walls/direction (realistic 14" walls, no
# shear overstress) and puts stiffness at the perimeter (kills the torsion).
WALLS = [
   ((0,0),(24,0),"PX1"),   ((72,0),(96,0),"PX2"),     # grid 1 (Y=0)   resist X
   ((0,72),(24,72),"PX3"), ((72,72),(96,72),"PX4"),   # grid 4 (Y=72)  resist X
   ((0,0),(0,24),"PY1"),   ((0,48),(0,72),"PY2"),     # grid A (X=0)   resist Y
   ((96,0),(96,24),"PY3"), ((96,48),(96,72),"PY4"),   # grid E (X=96)  resist Y
]
piers = set()
nw = 0
for (p1, p2, pier) in WALLS:
   if pier not in piers:
      try: sm.PierLabel.SetPier(pier)
      except Exception as e: step(f"SetPier {pier} flagged: {e}")
      piers.add(pier)
   for s in range(NSTORY):
      zb, zt = ZALL[s], ZALL[s+1]
      X4 = [p1[0], p2[0], p2[0], p1[0]]
      Y4 = [p1[1], p2[1], p2[1], p1[1]]
      Z4 = [zb,    zb,    zt,    zt  ]
      nm = add_area(X4, Y4, Z4, "WALL14")
      try: sm.AreaObj.SetPier(nm, pier)
      except Exception: pass
```

```
        nw += 1
step(f"Perimeter wall panels added: {nw}  (piers: {len(piers)})")

# ------------------------------------------------------------------ floor slabs + diaphragms
try:
    for s in range(NSTORY):
        sm.Diaphragm.SetDiaphragm(f"D{s+1}", False)        # rigid
except Exception as e:
    step("Diaphragm define flagged: "+str(e))

slab_by_floor = {s: [] for s in range(NSTORY)}
nslab = 0
for s in range(NSTORY):
    z = ZT[s]
    for i in range(len(XS)-1):
        for j in range(len(YS)-1):
            X4=[XS[i],XS[i+1],XS[i+1],XS[i]]
            Y4=[YS[j],YS[j],YS[j+1],YS[j+1]]
            Z4=[z,z,z,z]
            nm = add_area(X4,Y4,Z4,"SLAB8"); slab_by_floor[s].append(nm); nslab += 1
step(f"Floor slab panels added: {nslab}")

for s in range(NSTORY):
    for nm in slab_by_floor[s]:
        try: sm.AreaObj.SetDiaphragm(nm, f"D{s+1}")
        except Exception: pass
```

```
step("Rigid diaphragm assigned per floor")


# ------------------------------------------------------------------ base restraints
sm.SetModelIsLocked(False)
try:
    nn, pts, ret = sm.PointObj.GetNameList()
    fixed = 0
    for p in pts:
        xyz = sm.PointObj.GetCoordCartesian(p, 0,0,0)
        z = xyz[2]
        if abs(z) < 1e-4:
            sm.PointObj.SetRestraint(p, [True,True,True,True,True,True]); fixed += 1
    step(f"Base joints fixed: {fixed}")
except Exception as e:
    step("Base restraint loop flagged: "+str(e))


# ------------------------------------------------------------------ load patterns
#  Add(Name, Type, SelfWtMult, AddCase)  Type: 1 Dead, 2 SuperDead, 3 Live, 5 Quake
sm.LoadPatterns.Add("DEAD", 1, 1.0, True)
sm.LoadPatterns.Add("SDL",  2, 0.0, True)
sm.LoadPatterns.Add("LIVE", 3, 0.0, True)
sm.LoadPatterns.Add("LROOF",3, 0.0, True)
sm.LoadPatterns.Add("EQX",  5, 0.0, True)
sm.LoadPatterns.Add("EQY",  5, 0.0, True)
step("Load patterns: DEAD, SDL, LIVE, LROOF, EQX, EQY")
```

```
# area loads (Dir 10 = gravity, +value downward)
for s in range(NSTORY):
    roof = (s == NSTORY-1)
    sdl  = SDL_ROOF if roof else SDL_FLR
    liv  = LIVE_ROOF if roof else LIVE_FLR
    lpat = "LROOF" if roof else "LIVE"
    for nm in slab_by_floor[s]:
        sm.AreaObj.SetLoadUniform(nm, "SDL",  sdl, 10, True, "Global")
        sm.AreaObj.SetLoadUniform(nm, lpat,   liv, 10, True, "Global")
step("Area loads applied (SDL + Live, roof reduced)")


# perimeter cladding line load
for nm in peri:
    try: sm.FrameObj.SetLoadDistributed(nm, "SDL", 1, 10, 0.0, 1.0, CLAD, CLAD, "Global", True,
True)
    except Exception: pass
step(f"Cladding line load {CLAD} k/ft applied to {len(peri)} perimeter beams")


# ------------------------------------------------------------------ mass source
# seismic weight W = DEAD + SDL (office: no live in W)
try:
    sm.PropMaterial  # noqa
    sm.MassSource.SetMassSource("MSSSRC", False, False, True, True,
                                2, ["DEAD","SDL"], [1.0,1.0])
    step("Mass source = 1.0 DEAD + 1.0 SDL")
except Exception as e:
```

```
    step("MassSource.SetMassSource flagged - set mass from DEAD+SDL in GUI: "+str(e))

# ---------------------------------------------------------------- modal case
try:
    sm.LoadCases.ModalEigen.SetCase("MODAL")
    sm.LoadCases.ModalEigen.SetNumberModes("MODAL", 12, 1)
    step("Modal case: 12 eigen modes")
except Exception as e:
    step("Modal setup flagged: "+str(e))

# ---------------------------------------------------------------- load combos
D  = [("DEAD",1.0),("SDL",1.0)]
def combo(name, items):
    sm.RespCombo.Add(name, 0)
    for pat,sf in items:
        sm.RespCombo.SetCaseList(name, 0, pat, sf)

Ev = 0.2*1.16     # 0.232  (SDS = 1.16)
rho = 1.3
combo("1.4D",           [("DEAD",1.4),("SDL",1.4)])
combo("1.2D+1.6L+0.5Lr", [("DEAD",1.2),("SDL",1.2),("LIVE",1.6),("LROOF",0.5)])
combo("1.2D+1.6Lr+1.0L", [("DEAD",1.2),("SDL",1.2),("LROOF",1.6),("LIVE",1.0)])
for d,s in [("EQX",1),("EQX",-1),("EQY",1),("EQY",-1)]:
    combo(f"(1.2+Ev)D+{'+' if s>0 else '-'}rhoE+L_{d}",
          [("DEAD",1.2+Ev),("SDL",1.2+Ev),(d,rho*s),("LIVE",1.0),("LROOF",1.0)])
    combo(f"(0.9-Ev)D+{'+' if s>0 else '-'}rhoE_{d}",
```

```
        [("DEAD",0.9-Ev),("SDL",0.9-Ev),(d,rho*s)])
step("Load combinations built (1.4D, 1.2D+1.6L, seismic w/ Ev & rho=1.3)")


# ------------------------------------------------------------------ save + (optional) run
sm.View.RefreshView(0, False)
print("-"*70)
print("MODEL BUILT.  Seismic to assign: EQX/EQY -> Modify Lateral Load ->")
print("  ASCE 7-22:  Ss=1.57  S1=0.60  Site D  R=7  Omega0=2.5  Cd=5.5  Ie=1.0  TL=12")
print("  (hand check: Cs = SDS/(R/Ie) = 1.16/7 = 0.166 ; V = 0.166 W)")
if RUN_ANALYSIS:
    sm.Analyze.RunAnalysis()
    print("Analysis complete.")
print("="*70)
```